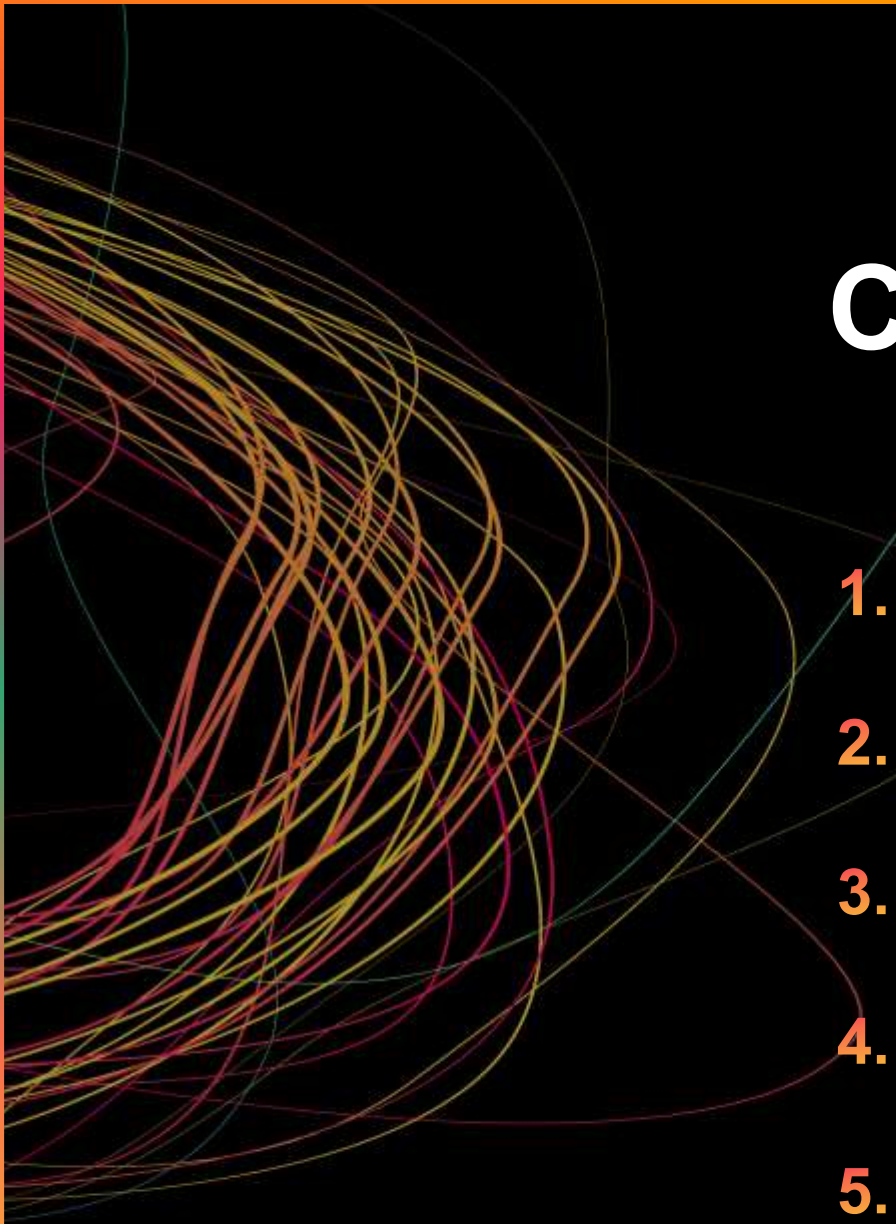


Emotion-Aware Music Visualization **via** a Lightweight GPT-Style Transformer Tagger



Catalogue

1. Abstract

2. Introduction

3. Background Survey

4. Method

5. Experiments

6. Case Study

7. Discussion and Limitations

8. Conclusion

9. Author Contributions

Reference

Abstract

- Traditional music visualization lacks deep integration of music emotion.
- Our method combines audio features, GPT-style lightweight Transformer labeling, and dynamic particle rendering.
- Experiments show high alignment with human perception and stable emotion-to-visual mapping.
- A small GPT-style Transformer effectively boosts emotional expressiveness and intelligence in visualization.

Introduction

Traditional visualization systems only analyze low-level audio signals, lacking the perception of high-level emotional semantics of music, which leads to rigid and inexpressive visual effects.

VS

GPT style Transformers have powerful semantic understanding capabilities, which can be used to realize intelligent emotion annotation for music.

Introduction

3 Main Contributions:

- GPT-style Transformers excel at sequence and context modeling: extract rhythm, harmony, chord, loudness → output emotion labels to web page.
- Emotion-to-particle mapping: unique glow color per emotion; particle size & dynamics sync with music volume/rhythm.
- Full-stack visualization system: integrates Python emotion recognition with web-based particle rendering for stable emotional audio-visual linkage.

The code is adapted from the excellent GPT tutorial created by Apoorv Nandan available on the Keras website.

Background Survey

- **Traditional visualization:**
converts low-level audio (volume, spectrum, rhythm) into simple geometric animations.
 - **Lack of emotion understanding:**
visual presentation disconnected from music's inner atmosphere → poor audiovisual experience.
 - **Early emotion recognition:**
relies on handcrafted features & shallow models (SVM, random forest).
 - **Label inconsistency:**
poor cross-dataset generalization → unstable emotion labels for high-quality visualization.
- ★ Traditional methods cannot support emotionally expressive and consistent music visualization.

Background Survey

LLMs overcome prior limitations via semantic understanding & contextual learning.

- LLMs capture emotional ambiguity & generate reasonable emotion distributions from audio features. (Hong et al. 2025)
- LLM embeddings enable cross-dataset emotion label alignment & zero-shot music emotion prediction. (Liu et al. 2024)
- Converting audio features into text prompts improves emotion recognition accuracy. (Chandraumakantham et al. 2024)

★ LLM-based emotion annotation enables visualization to match music emotion, breaking traditional monotony.

Method

3 representative low-level music features are extracted using Librosa

Tempo:

- Estimated beat-per-minute (BPM) through beat tracking.
- Reflects music speed and rhythm intensity.

Loudness:

- Calculated from RMS (root mean square) energy.
- Normalized into three levels: low, mid, high.

Chord & Tonality:

- Root note determined by maximum energy in chroma feature.
- Output as major or minor tonality.

★ **Feature Integration:** All features are discretized and formatted into a unified textual structure, which serves as the standard input for the emotion prediction model.

3 representative low-level music features are extracted using Librosa

Method - Code

```
from google.colab import drive
drive.mount('/content/drive')

import librosa
import pandas as pd
import os

TRAIN_FOLDER = "/content/drive/MyDrive/trainMusic"
CSV_PATH = "train_emo.csv"
OUTPUT_CSV = "train_features_with_emotion.csv"
```

```
# Find the corresponding audio file
found = False
for fname in os.listdir(TRAIN_FOLDER):
    if song_name in fname:
        full_path = os.path.join(TRAIN_FOLDER, fname)
        tempo, loudness, chord = extract_music_featu

        results.append([
            "Song Name": song_name,
            "tempo": tempo,
            "loudness": loudness,
            "chord": chord,
            "emotion": emotion
        ])
    print(f"✅ Extracted: {song_name}")
    found = True
    break
```

```
# 4. Save as final training table
df_out = pd.DataFrame(results)
df_out.to_csv(OUTPUT_CSV, index=False, encoding="utf-8-sig")

print(f"📁 Output file: {OUTPUT_CSV}")
print(df_out.head())
```

📁 Output file: train_features_with_emotion.csv

	Song Name	tempo	loudness	chord	emotion
0	A Thousand Years	139	mid	A# major	tender heartfelt
1	All About That Bass	89	mid	E major	confident sassy
2	Almost Lover	123	mid	C major	sorrowful regretful
3	Animals	95	high	E major	intense passionate

df.head(30)

```
import pandas as pd
df = pd.read_csv("train_features_with_emotion.csv")
```

	Song Name	tempo	loudness	chord	emotion
0	A Thousand Years	139	mid	A# major	tender heartfelt
1	All About That Bass	89	mid	E major	confident sassy
2	Almost Lover	123	mid	C major	sorrowful regretful
3	Animals	95	high	E major	intense passionate
4	Anti-Hero	96	mid	E major	introspective vulnerable
5	As Long As You Love Me	140	high	C major	devoted passionate

Method

GPT Style Lightweight Transformer Tagger (adapted from Keras)

Model task

- **Input:** Structured music feature text
- **Output:** Corresponding emotional labels (such as powerful warm)
- **Goal:** Learn the mapping from acoustic features to emotions

Model Structure - Three Core Components

- **TextVectorization:** Converting text features into integer sequences
- **TokenAndPositionEmbedding:** Injecting positional information to maintain sequence order
- **Transformer block** (multi head causal attention): Capturing long-range dependencies between features and emotions

Training method

- Generate reasonable and consistent emotion predictions using paired datasets (music features + corresponding emotion labels)

GPT Style Lightweight Transformer Tagger (adapted from Keras)

Method – Code

| Causal attention mask function

```
# Display the same example converted to ints
example_tokenised = vectorize_layer(example_data)
print(example_tokenised.numpy())

[ 6  9  4  5  2 96  3  8  2 14  3 11  2 15 12  7  3 10  4 20 70  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0]
```

```
# Create training sequences (input = previous tokens,
def prepare_inputs(text):
    text = tf.expand_dims(text, -1)
    tokenized_sentences = vectorize_layer(text)
    x = tokenized_sentences[:, :-1]
    y = tokenized_sentences[:, 1:]
    return x, y

train_ds = text_ds.map(prepare_inputs)
example_input_output = train_ds.take(1).get_single_element()
```

```
def causal_attention_mask(batch_size, n_dest, n_src, dtype):
    """
    batch_size: Number of sequences in the batch
    n_dest: Number of destination tokens (sequence length for output).
    n_src: Number of source tokens (sequence length for input).
    dtype: Data type of the mask (e.g., tf.int32, tf.float32).
    """

    i = tf.range(n_dest)[: , None]
    j = tf.range(n_src)
    m = i >= j - n_src + n_dest # Creates a boolean matrix
    mask = tf.cast(m, dtype) # Converts True to 1, False to 0
    mask = tf.reshape(mask, [1, n_dest, n_src]) # add one dimension
    # Repeats the mask along the batch dimension
    mult = tf.concat( # create a tensor looks like [batch_size, 1, 1], this tensor
        [tf.expand_dims(batch_size, -1), tf.constant([1, 1], dtype=tf.int32)], 0
    )
    return tf.tile(mask, mult) # repeat mask with batch_size times

np.transpose(causal_attention_mask(1, 10, 10, dtype=tf.int32)[0]) # transpose the result

array([[1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
       [0, 1, 1, 1, 1, 1, 1, 1, 1, 1],
       [0, 0, 1, 1, 1, 1, 1, 1, 1, 1],
       [0, 0, 0, 1, 1, 1, 1, 1, 1, 1],
       [0, 0, 0, 0, 1, 1, 1, 1, 1, 1],
       [0, 0, 0, 0, 0, 1, 1, 1, 1, 1],
       [0, 0, 0, 0, 0, 0, 1, 1, 1, 1],
       [0, 0, 0, 0, 0, 0, 0, 1, 1, 1],
       [0, 0, 0, 0, 0, 0, 0, 0, 1, 1],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 1]], dtype=int32)
```

GPT Style Lightweight Transformer Tagger (adapted from Keras)

Method – Code

| Transformer Block Layer

```
class TransformerBlock(layers.Layer):
    def __init__(self, num_heads, key_dim, embed_dim, ff_dim, dropout_rate=0.1):
        super(TransformerBlock, self).__init__()
        self.num_heads = num_heads
        self.key_dim = key_dim
        self.embed_dim = embed_dim
        self.ff_dim = ff_dim
        self.dropout_rate = dropout_rate
        self.attn = layers.MultiHeadAttention(num_heads=self.num_heads, key_dim=key_dim)
        self.dropout_1 = layers.Dropout(self.dropout_rate)
        self.ln_1 = layers.LayerNormalization(epsilon=1e-6)
        self.ffn_1 = layers.Dense(self.ff_dim, activation="relu")
        self.ffn_2 = layers.Dense(self.embed_dim)
        self.dropout_2 = layers.Dropout(self.dropout_rate)
        self.ln_2 = layers.LayerNormalization(epsilon=1e-6)
```

```
# The call method defines the forward pass of the layer
def call(self, inputs):
    input_shape = tf.shape(inputs)
    batch_size = input_shape[0]
    seq_len = input_shape[1]
    # Generate a causal mask
    causal_mask = causal_attention_mask(
        batch_size, seq_len, seq_len, tf.bool
    )
    # Compute attention scores and outputs using
    attention_output, attention_scores = self.attn(
        inputs,
        inputs,
        attention_mask=causal_mask,
        return_attention_scores=True,
    )
```

```
# Gets the configuration of the layer. The same layer
def get_config(self):
    config = super().get_config()
    config.update(
        {
            "key_dim": self.key_dim,
            "embed_dim": self.embed_dim,
            "num_heads": self.num_heads,
            "ff_dim": self.ff_dim,
            "dropout_rate": self.dropout_rate,
        }
    )
    return config
```

```
class TokenAndPositionEmbedding(layers.Layer):
    def __init__(self, max_len, vocab_size, embed_dim):
        super().__init__()
        self.max_len = max_len
        self.vocab_size = vocab_size
        self.embed_dim = embed_dim
        # 0用作padding, 自动生成序列mask
        self.token_emb = layers.Embedding(vocab_size, embed_dim)
        self.pos_emb = layers.Embedding(max_len, embed_dim)

    def call(self, x):
        L = tf.shape(x)[-1]
        pos = tf.range(L)
        pos = self.pos_emb(pos)
        x = self.token_emb(x)
        return x + tf.expand_dims(pos, 0)

    # 继续向后游传tokens的padding mask
    def compute_mask(self, inputs, mask=None):
        return self.token_emb.compute_mask(inputs)

    def get_config(self):
        return {"max_len": self.max_len, "vocab_size": self.vocab_size, "embed_dim": self.embed_dim}
```


Method – Code — Transformer model

GPT Style Lightweight Transformer Tagger (adapted from Keras)

```
inputs = layers.Input(shape=(None,), dtype=tf.int32)
x = TokenAndPositionEmbedding(max_len=80, vocab_size=len(vocab), embed_dim=256)(inputs)
x, attention_scores = TransformerBlock(
    num_heads=4, key_dim=256, embed_dim=256, ff_dim=256
)(x)
outputs = layers.Dense(len(vocab), activation="softmax")(x)
gpt = models.Model(inputs=inputs, outputs=[outputs, attention_scores])
gpt.compile("adam", loss=[losses.SparseCategoricalCrossentropy(), None])

gpt.summary()
```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/layer.py:982: UserWarning: Layer 'input_layer_1' with a mask attached to it. However, this layer does not support masking and will therefore ignore the mask. warnings.warn(
Model: "functional_1"

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, None)	0
token_and_position_embedding_1 (TokenAndPositionEmbedding)	(None, None, 256)	48,128
transformer_block_1 (TransformerBlock)	[(None, None, 256), (None, 4, None, None)]	1,184,512
dense_5 (Dense)	(None, None, 108)	27,756

Total params: 1,260,396 (4.81 MB)
Trainable params: 1,260,396 (4.81 MB)
Non-trainable params: 0 (0.00 B)

```
def sample_from(self, probs, temperature=None, top_k=None):
    # 温度与top_k采样
    t = float(self.default_temperature if temperature is None else temperature)
    t = max(1e-3, t) # 避免除零/数值不稳
    p = np.asarray(probs, dtype=np.float64)

    # top-k过滤
    k = self.top_k if top_k is None else top_k
    if k is not None and 0 < k < p.size:
        idx = np.argpartition(-p, k)[:k]
        mask = np.zeros_like(p)
        mask[idx] = p[idx]
        p = mask

    # 温度重标定
    p = p ** (1.0 / t)
    s = p.sum()
    if not np.isfinite(s) or s <= 0:
        p = np.ones_like(p) / p.size
    else:
        p /= s
```

```
# Tokenize starting prompt
text_generator = TextGenerator(vocab)

gpt.fit(
    train_ds,
    epochs=5,
    callbacks=[text_generator],
```

Method

Prompt Construction and Inference

Construct a structured prompt which from the extracted music features:

music feature: tempo= tempo, loudness= loudness, chord= chord, emotion:

The model performs autoregressive generation to complete the emotion label, which ensures the output follows the predefined format and maintains the consistency with the training distribution.

Method

Robust Parsing and Rule-Based Fallback

Two-step parsing strategy

- Use regular expression to extract predicted label after “*emotion:*”
- If parsing fails → activate rule-based fallback

Nature of the fallback

- Serves only as a temporary safeguard
- Provides only coarse labels (cannot capture subtle audio–emotion correlations)

Actual performance

- Trained GPT-style Transformer significantly outperforms rule mapping in plausibility and consistency
- Most remaining errors are out-of-distribution (OOD) mispredictions (unseen music styles), not typical LLM hallucinations

Method - Code

Robust Parsing and Rule-Based Fallback

```
tempo, loudness, chord = extract_music_features(filename)

print(f"• Tempo: {tempo} BPM")
print(f"• Loudness: {loudness}")
print(f"• Chord: {chord}")

/tmp/ipykernel_18397/464291953.py:7: DeprecationWarning: Convert
extract a single element from your array before performing this
    tempo = int(tempo)
• Tempo: 99 BPM
• Loudness: high
• Chord: A# major
```

```
return emotion

predicted_emotion = predict_emotion_from_features(tempo, loudness, chord)

generated text:
music feature: tempo=99, loudness=high, chord=A# major, emotion: loudness = hi

✅ GPT model prediction successful!
```

```
✅ Tempo: 99 BPM
✅ Loudness: high
✅ Chord: A# major
🚫 Predicted emotion: 【 loudness neutral 】
```

```
with open("emotion.txt", "w", encoding="utf-8") as f:
    f.write(predicted_emotion)

print("\n✅ The file has been saved: 【emotion.txt】 → The webpage can
```

```
✅ The file has been saved: 【emotion.txt】 → The webpage can be read directly.
```

```
def predict_emotion_from_features(tempo, loudness, chord):
    emotion = None

    # Step 1: Prioritize using the GPT model for prediction
    try:
        prompt = f"music feature: tempo={tempo}, loudness={loudness}, chord={chord}, emotion:"
        info = text_generator.generate(prompt, max_tokens=20, temperature=0.5, top_k=3)
        generated_text = info[-1]["prompt"]

        match = re.search(r"emotion:\s*([\w\s]+?) (?=\s*(?:music feature:|$))", generated_text, re.DOTALL)
        if match:
            emotion_raw = match.group(1).strip()
            words = emotion_raw.split()

            invalid_words = ["loudness", "tempo", "chord", "music", "feature", "emotion"]
            if any(bad in emotion_raw for bad in invalid_words):
                print("⚠️ GPT output is invalid feature word → skip model")
                raise Exception("invalid output")

            # Normal emotions
            if len(words) >= 2:
                emotion = f"{words[0]} {words[1]}"
            elif len(words) == 1:
                emotion = emotion_raw
            else:
                raise Exception("empty emotion")

            if emotion:
                print("✅ GPT model prediction successful!")
                return emotion
        except:
            print("\n⚠️ GPT prediction failed.")
```

Experiments

Dataset

- **Self-built dataset:** 50 music clips from online platforms, manually annotated with emotion categories (happy, energetic, powerful, warm, calm, sad, etc.) following common streaming platform classifications.
- **38** for training, **12** for testing: Small scale, limited style coverage.

Evaluation Metrics (Two subjective evaluation metrics consistent with music emotion's natural ambiguity)

- **Emotion Label Accuracy:** whether predicted emotion matches human perception.
- **Visual-Emotion Consistency:** 1–5 scale (1=strongly inconsistent, 5=highly consistent) evaluating particle color & dynamics against music's emotional atmosphere.

Experiments

Results

- **User test:** 5 users rated 12 clips (1–5 scale) on visual-emotion consistency.
- **Acceptance criterion:** ≥ 3 out of 5 users giving score ≥ 4 (“somewhat/highly consistent”).
- **Result:** 10/12 music ($\approx 83\%$) achieved majority acceptance.

Analysis

- **Good performance:** on styles similar to training data – small dataset learns obvious patterns within limited distribution.
- **Poor predictions:** on stylistically different or emotionally vague music – due to low dataset diversity, limited generalization, and distribution shift.

Case Study

“Tong Hua Shun” 《同花顺》 (Chinese pop ballad)

—— mid-tempo, warm, similar acoustic structure to training set.

- **Features extracted via Librosa:** stable rhythm, medium-low loudness, C# major.
- **Result:** Transformer outputs “powerful warm” → warm-color particles with medium rhythm, smooth motion, matching the changing atmosphere.

★ Conclusion:

High accuracy due to acoustic similarity (not language) → good cross-lingual generalization for emotion-aware visualization.

Case Study - Code

🎵 Please upload your audio file (mp3 / wav)

选择文件 同花顺.mp3

同花顺.mp3(audio/mpeg) - 3808535 bytes, last modified: 2026/4/15 - 100% done
Saving 同花顺.mp3 to 同花顺.mp3

✅ Upload: 同花顺.mp3

```
# extract features
```

```
tempo, loudness, chord = extract_music_features(filename)
```

```
print(f"• Tempo: {tempo} BPM")
```

```
print(f"• Loudness: {loudness}")
```

```
print(f"• Chord: {chord}")
```

/tmp/ipykernel_18397/464291953.py:7: DeprecationWarning: Conversion of an array-like object into a scalar and extracting a single element from your array before performing this operation.

```
tempo = int(tempo)
```

- Tempo: 125 BPM

- Loudness: low

- Chord: C# major

✅ Table lookup successful!

/tmp/ipykernel_18397/861681089.py:43: SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame.

Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: <https://pandas.pydata.org/pandas-docs/stab>

```
candidates["tempo_diff"] = abs(candidates["tempo"] - tempo)
```

```
# Step 2: Look up the table to find the most similar training set
try:
    df_train = pd.read_csv("train_features_with_emotion.csv")
    candidates = df_train[
        (df_train["loudness"] == loudness) &
        (df_train["chord"] == chord)
    ]
    if len(candidates) > 0:
        candidates["tempo_diff"] = abs(candidates["tempo"] - tempo)
        best = candidates.sort_values("tempo_diff").iloc[0]
        print("✅ Table lookup successful!")
        return best["emotion"]
except:
    print("⚠️ Table lookup failed.")
```

✅ Tempo: 125 BPM

✅ Loudness: low

✅ Chord: C# major

🌀 Predicted emotion: 【 somber wistful 】

```
with open("emotion.txt", "w", encoding="utf-8") as f:
    f.write(predicted_emotion)
```

```
print("\n✅ The file has been saved: 【emotion.txt】 → The webpage c
```

✅ The file has been saved: 【emotion.txt】 → The webpage can be read directly.

Case Study - Code

js.js > wordColor > catchy

```
1 // Emotional word color library
2 const wordColor = {
3   // Bright
4   powerful: "255,180,80",
5   warm: "255,220,160",
6   happy: "255,230,100",
7   energetic: "255,210,0",
8   cheerful: "255,240,120",
9   optimistic: "255,235,140",
10  exuberant: "255,200,80",
11  youthful: "255,220,120",
12  vibrant: "255,190,60",
13  liberating: "255,180,100",
14  uplifting: "255,210,120",
15  empowering: "255,170,90",
16  confident: "255,160,100",
17  bold: "255,140,60",
18  sassy: "255,150,120",
19  bubbly: "255,240,160",
20  giddy: "255,230,140",
21  carefree: "240,255,160",
22  lighthearted: "230,255,180",
23  upbeat: "255,220,100",
24  playful: "255,235,120",
```

```
68 // Automatically read emotion.txt and mix colors automatically
69 let emotion = "powerful warm";
70 let color = {
71   core: wordColor.powerup + "," + wordColor.warm,
72   glow: wordColor.powerup + "," + wordColor.warm
73 };
74
75 function mixColor(c1, c2) {
76   let [r1,g1,b1] = c1.split(",").map(Number);
77   let [r2,g2,b2] = c2.split(",").map(Number);
78   let r = Math.round((r1 + r2) / 2);
79   let g = Math.round((g1 + g2) / 2);
80   let b = Math.round((b1 + b2) / 2);
81   return `${r},${g},${b}`;
82 }
83
84 fetch("emotion.txt")
85   .then(res => res.text())
86   .then(text => {
87     emotion = text.trim();
88     document.getElementById("emotion").innerText = emotion;
89     let words = emotion.split(/\s+/).filter(w => w);
90     let w1 = words[0] || "powerful";
91     let w2 = words[1] || "warm";
92     let c1 = wordColor[w1] || wordColor.powerful;
93     let c2 = wordColor[w2] || wordColor.warm;
94     color.core = mixColor(c1, c2);
95     color.glow = mixColor(c1, c2);
96   })
97   .catch(() => {
98     document.getElementById("emotion").innerText = emotion;
99   });
```

emotion.txt
1 somber wistful

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <title>Music Emotion Visualization</title>
6   <link rel="stylesheet" href="css.css">
7 </head>
8 <body>
9   <div id="particles"></div>
10  <h2 id="emotion"></h2>
11  <audio id="audio" controls src="testMusic/同花顺.mp3"></audio>
12
13  <script src="js.js"></script>
14 </body>
15 </html>
```

7045MINIPROJ-2

> .vscode

testMusic

- 才二十三.mp3
- 舍得.mp3
- 圣诞快乐劳伦斯先生.mp3
- 同花顺.mp3
- 小幸运.mp3
- Cruel Summer.mp3
- Love Story.mp3
- My Sunshine.mp3

The predict saved in [emotion.txt] → Webpage can be read directly.

Discussion and Limitations

4 Limitations:

- **Small dataset**: only English songs → limited stylistic & linguistic diversity.
- **Oversimplified audio features**: loudness lacks expressive dynamics; chord only major/minor; BPM prone to errors.
- **Ignoring lyrics**: inconsistency when audio and lyrical emotions conflict (e.g., energetic rock + sorrowful lyrics).
- **Fixed emotion-to-particle mapping**: manual & static → limits visual richness, cannot fully match music's emotional experience.

Discussion and Limitations

3 Future improvements:

- Larger-scale multilingual **dataset**.
- Dedicated **music-domain** GPT-style Transformer.
- **Audio–lyrics** multimodal fusion to enhance emotion accuracy.
- **CNN-based learning** to automatically map emotion features to particle parameters (color, speed, size, density) – replacing manual rules.

Conclusion

- **Lightweight system**: audio features (tempo, loudness, chord) + GPT-style Transformer + particle rendering.
- **Fallback mechanism** ensures stable output.
- **Experiments**: small GPT Transformer captures high-level emotion semantics, outperforming traditional audio-only visualization.
- **Applications**: feasible for digital art, HCI, small-stage performance — an effective solution for emotion-aware music visualization.

Author Contributions

The project was completed independently by Yin Hanyu.

- Proposing the idea and designing the overall system architecture;
 - Collecting and annotating the dataset;
 - Implementing the audio feature extraction, Transformer-based emotion prediction model, and particle rendering pipeline;
 - Conducting all experiments and case studies;
 - Analyzing the results and drafting the full manuscript;
 - Preparing the presentation slides.
-
- The Transformer implementation was adapted from the **Keras 'miniature GPT' tutorial by Apoorv Nandan**;
 - All task-specific data processing, training, evaluation, and visualization code were designed and implemented by YIN Hanyu.

Reference

1. O. Chandraumakantham, N. Gowtham, M. Zakariah, and A. Almazyad, "Multimodal Emotion Recognition Using Feature Fusion: An LLM-Based Approach," **IEEE Access**, vol. 12, pp. 108052–108071, 2024, doi: 10.1109/ACCESS.2024.3425953.
2. X. Hong, Y. Gong, V. Sethu, and T. Dang, "AER-LLM: Ambiguity-Aware Emotion Recognition Leveraging Large Language Models," in **ICASSP 2025**, Hyderabad, India, 2025, pp. 1–5, doi: 10.1109/ICASSP49660.2025.10888198.
3. S. Wang, T. Ouyang, Y. Zhou, Q. Xiao, Y. Ren, Y. Pan, and C. Luo, "Enhanced Emotion-Aware Music Recommendation via Large Language Models," in **Proc. 31st ACM SIGKDD Conf. Knowl. Discov. Data Mining**, vol. 2, pp. 4986–4994, 2025.
4. R. Liu, A. Roy, and D. Herremans, "Leveraging LLM Embeddings for Cross Dataset Label Alignment and Zero Shot Music Emotion Prediction," **arXiv preprint**, arXiv:2410.11522, 2024.
5. Y. Xia and F. Xu, "Study on Music Emotion Recognition Based on the Machine Learning Model," **Math. Problems Eng.**, vol. 2022, pp. 1–11, 2022, doi: 10.1155/2022/9256586.
6. J. Cathy and S. Pradeepkumar, "Enhancing Emotional Intelligence in LLM Responses Using Speech Emotion Recognition," **arXiv preprint**, arXiv:2503.04567, 2025.
7. D. Foster, "Generative Deep Learning, 2nd ed.—Transformer notebooks," GitHub repository, 2023. Accessed: Apr. 16, 2026. [Online]. Available: https://github.com/davidADSP/Generative_Deep_Learning_2nd_Edition/tree/main/notebooks/09_transformer/gpt
8. A. Nandan, "Text generation with a miniature GPT," Keras Code Examples, 2023. Accessed: Apr. 16, 2026. [Online]. Available: https://keras.io/examples/generative/text_generation_with_miniature_gpt/

Emotion-Aware Music Visualization **via** a Lightweight GPT-Style
Transformer Tagger

THANK YOU FOR WATCHING